Apuntes · Clase 46 · Autoencoders y representaciones

Para leer antes de los ejercicios · pensado para quien acaba de aprender redes neuronales y nunca ha visto un autoencoder

Lo más importante, en 8 ideas

1. **Autoencoder.** red entrenada para reproducir su propia entrada pasando por un resumen pequeño.
2. **Encoder y decoder.** $h = f(x)$ resume (64 → 8); $\hat{x} = g(h)$ reconstruye (8 → 64).
3. **Referencia = entrada.** no hacen falta etiquetas; la pérdida compara x con $\hat{x}$.
4. **MSE por píxel.** promedio de $(\hat{x}_j - x_j)^2$; ℓ por imagen, L por conjunto.
5. **Resumir es perder.** mismo h implica misma reconstrucción; 8 reales no son 8 bits.
6. **Referencias.** imagen media y PCA 8: un MSE se entiende al compararlo.
7. **Parámetros.** unidades × (entradas + 1) por capa; latente y parámetros no son lo mismo.
8. **Alerta por error.** $e(x) > c$: es un puntaje, no una probabilidad; exige validación específica.

Cada sección trae una explicación en palabras simples, un **ejemplo resuelto con números distintos** a los de las diapositivas y los ejercicios, y un resumen «para recordar». Ejercicios en `C46_Ejercicios_Cuaderno_Autoencoders.pdf` y notebook `C46_Autoencoders.ipynb`.

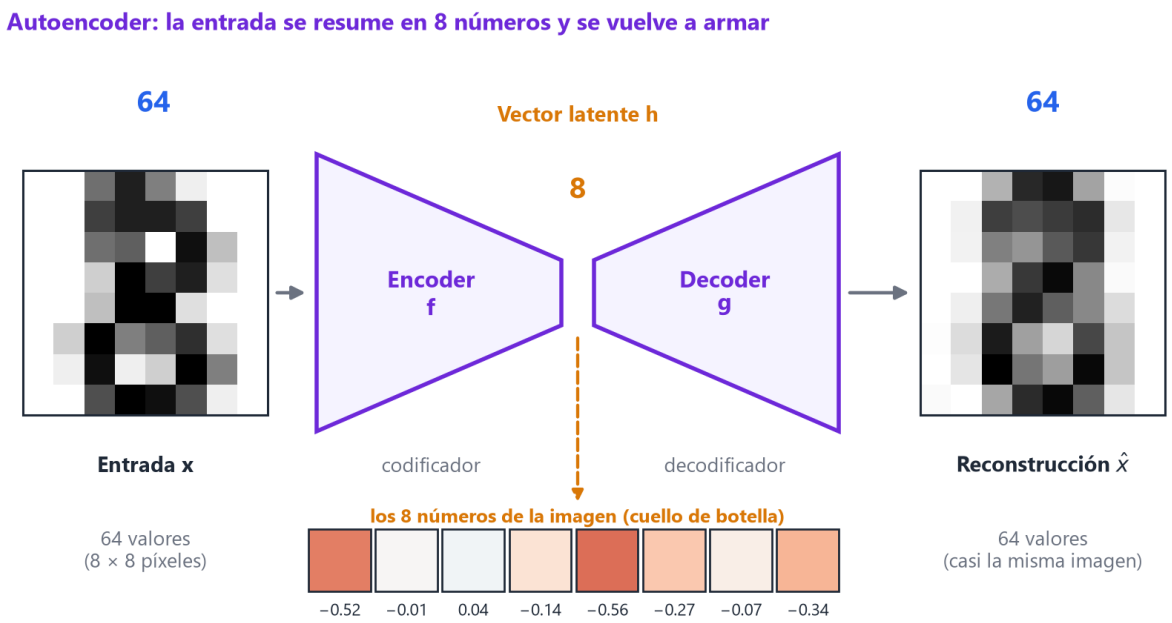
1 Qué es una representación y qué es un autoencoder

Una **representación** es una forma de describir un objeto con números. Una imagen de 8 × 8 se describe con 64 intensidades (de 0, blanco, a 1, negro, tras dividir por 16). Una red neuronal aprende representaciones en sus capas intermedias: cada capa recibe una descripción y entrega otra. Un **autoencoder** es una red con una forma especial: recibe x, la lleva a un **vector latente** h más pequeño con el **encoder** ($h = f(x)$) y desde h intenta estimar x otra vez con el **decoder** ($\hat{x} = g(h)$). No se pide que sea idéntico: solo que sea una aproximación buena. Y no se necesitan etiquetas: la referencia con la que se compara es la propia imagen.

Ejemplo resuelto · imagen de 4 píxeles → 2 números → 4 píxeles (ejemplo propio)

Paso	Cálculo	Resultado
Encoder (promedia de a dos)	$h_1 = (0 + 0.2)/2$; $h_2 = (0.9 + 1)/2$	$h = (0.10; 0.95)$
Decoder (repite cada h dos veces)	$\hat{x} = (h_1, h_1, h_2, h_2)$	$\hat{x} = (0.10; 0.10; 0.95; 0.95)$
Diferencias al cuadrado	$(0.1 - 0)^2$; $(0.1 - 0.2)^2$; $(0.95 - 0.9)^2$; $(0.95 - 1)^2$	0.0100; 0.0100; 0.0025; 0.0025
MSE de la imagen	$(0.0100 + 0.0100 + 0.0025 + 0.0025) \div 4$	0.00625
Otra imagen: (0; 1; 0; 1)	$h = (0.5; 0.5) \rightarrow \hat{x} = (0.5; 0.5; 0.5; 0.5)$	MSE = 0.25

Lectura: este resumen conserva bien una imagen «suave» (MSE 0.00625) y destruye una con contrastes entre píxeles vecinos (MSE 0.25). Un resumen solo conserva lo que su diseño permite conservar. **Error común**: pensar que el resumen es «una versión chica de la imagen»; es un conjunto de números cuyo significado lo decide el entrenamiento.



Ambas mitades aprenden juntas con un solo error: qué tan distinta quedó x̂ de x. La etiqueta del dígito no interviene.

Lo que debes recordar

- Autoencoder: $x \to h = f(x) \to \hat{x} = g(h)$; h es el vector latente (aquí 8 valores)
- La referencia es la entrada: no se usan etiquetas
- Las coordenadas latentes no tienen un significado individual

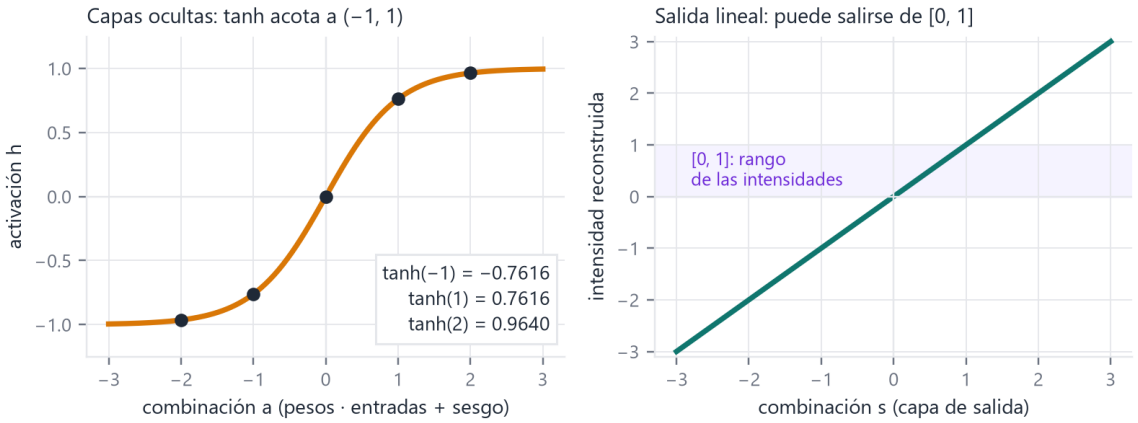
2 Un autoencoder pequeño, calculado a mano (tanh y salida lineal)

En la clase las capas ocultas usan **tanh** y la salida es **lineal** (sin activación). La tangente hiperbólica lleva cualquier número al intervalo (−1, 1): tanh(0) = 0, tanh(1) = 0.7616, tanh(−2) = −0.9640. La salida lineal puede dar valores fuera de [0, 1]; el MSE usa esas salidas completas y solo la visualización las recorta. Con una convención fija (cada imagen es una **fila** y se multiplica x·W), un autoencoder 3 → 2 → 3 se calcula así.

Ejemplo resuelto · red 3 → 2 → 3, sesgos en 0 (ejemplo propio)

Paso	Cálculo	Resultado
Combinación del encoder	$a = xW_1$ con x = (1; 0; 0.5), columnas (1, 0, 1) y (0, 1, −1)	a = (1.5; −0.5)
Vector latente	h = tanh(a)	h = (0.9051; −0.4621)
Salida lineal	$\hat{x} = hW_2$: $0.5h_1$; $0.5h_2$; $0.5h_1 - h_2$	$\hat{x}$ = (0.4526; −0.2311; 0.9147)
Cuadrados	$(\hat{x} - x)^2$ píxel a píxel	0.2997; 0.0534; 0.1720
MSE de la imagen	(0.2997 + 0.0534 + 0.1720) ÷ 3	0.1750
Parámetros	encoder 3·2 + 2; decoder 2·3 + 3	8 + 9 = 17

Lectura: aunque los sesgos valgan 0 aquí, se cuentan como parámetros porque se aprenden. **Error común**: aplicar tanh a la salida; en la receta de la clase la salida es lineal. Y recuerda que h tiene 2 números para una imagen de 3: ya se perdió información.



Lo que debes recordar

- Capas ocultas: tanh acota a (−1, 1); salida: lineal, puede salirse de [0, 1]
- Cada imagen es una fila: a = xW + b; h = tanh(a); $\hat{x} = hW' + b'$
- Los sesgos también son parámetros

3 El error de reconstrucción (MSE): por píxel, por imagen y por conjunto

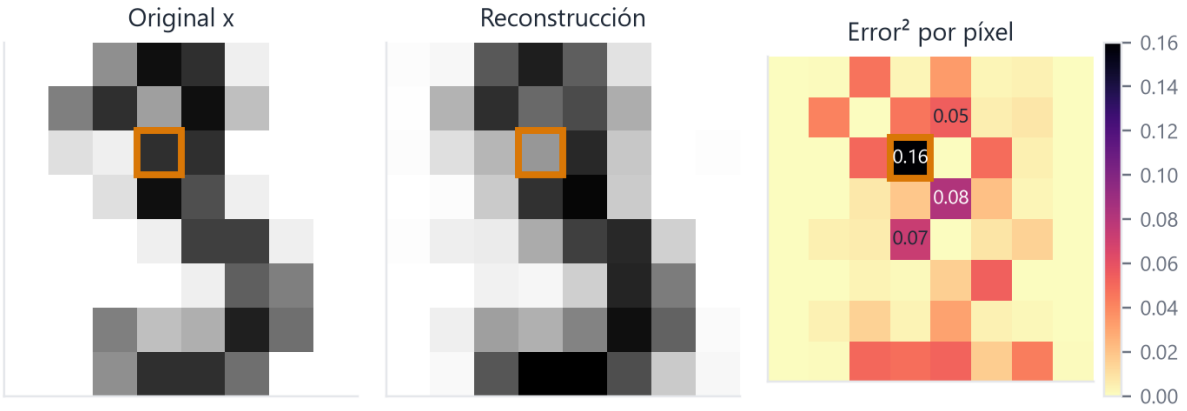
Para cada píxel se resta la entrada a la reconstrucción, se eleva al cuadrado, y se promedian los 64 cuadrados para obtener el error de la **imagen**: $\ell(x, \hat{x}) = (1/64)\ \Sigma_j\ (\hat{x}_j - x_j)^2$. El error del **conjunto** es el promedio de los errores de sus imágenes. Se eleva al cuadrado para que los signos no se cancelen y para penalizar más los errores grandes. El precio: el MSE solo dice «cuánto», no «hacia dónde» se equivocó. Un MSE de 0 es una copia perfecta, que no resume nada: no es la meta.

Ejemplo resuelto · imagen de 4 píxeles y su reconstrucción (ejemplo propio)

Píxel	Entrada x	Reconstrucción $\hat{x}$	Diferencia	Al cuadrado
1	0.00	0.10	0.10	0.0100
2	0.50	0.50	0.00	0.0000
3	1.00	0.60	−0.40	0.1600
4	0.25	0.35	0.10	0.0100
MSE				0.1800 ÷ 4 = 0.0450

Lectura: el píxel 3 se equivocó por 0.4 y aporta 0.16 de los 0.18 totales: el cuadrado hace que un error grande domine. En la clase: el píxel de la diapositiva 5 vale 0.8125 y se reconstruye como 0.4089: $(-0.4036)^2 = 0.1629$; los 64 cuadrados de esa imagen suman 1.1021 y su promedio es 0.017220. **Error común**: promediar las diferencias sin elevarlas al cuadrado (los signos se cancelan).

Dígito 3 (id 4): MSE de la imagen = promedio de los 64 cuadrados = 0.017220



Cuadro naranja (fila 3, columna 4): $(0.4089 - 0.8125)^2 = (-0.4036)^2 = 0.1629$

Lo que debes recordar

- ℓ = promedio de $(\hat{x}_j - x_j)^2$ sobre los 64 píxeles; L = promedio sobre las imágenes
- −0.3 y +0.3 cuestan lo mismo (0.09): el MSE no dice la dirección
- Error 0 = copia perfecta: no es la meta

4 Referencias: imagen media y PCA con 8 componentes

Un MSE aislado no dice si es bueno; hay que compararlo. La **imagen media** reconstruye todas las imágenes con el promedio de entrenamiento (no usa la entrada). **PCA** con k componentes guarda k números por imagen: $z = (x - \bar{x})P_k$, y reconstruye $\hat{x} = \bar{x} + zP_k^T$; es una reconstrucción **lineal**. Ambas se ajustan solo con entrenamiento. El autoencoder tiene la misma cantidad de números en el resumen, pero una reconstrucción **no lineal**.

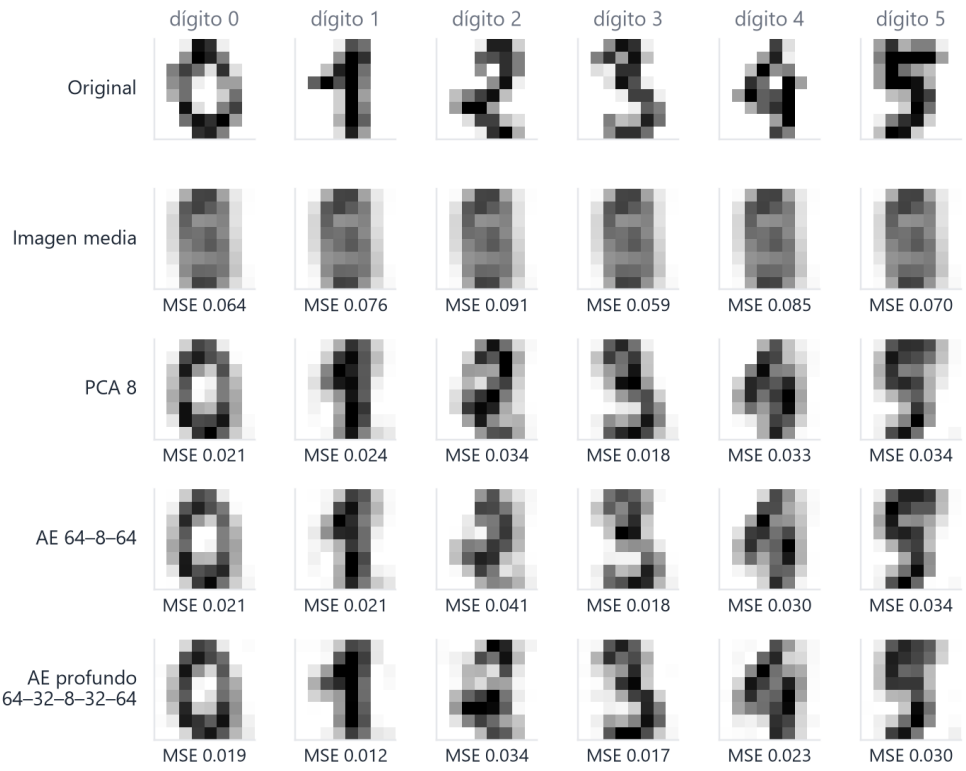
Ejemplo resuelto · PCA con 1 componente en 2 píxeles (ejemplo propio)

Paso	Cálculo	Resultado
Centrar	$x - \bar{x} = (0.9 - 0.5; 0.7 - 0.5)$	(0.4; 0.2)
Código z	$(0.4 + 0.2) \div \sqrt{2}$, con $P_1 = (1/\sqrt{2}; 1/\sqrt{2})$	z = 0.4243
Reconstruir	$\bar{x} + z \cdot P_1 = (0.5 + 0.3; 0.5 + 0.3)$	$\hat{x} = (0.8; 0.8)$
MSE de PCA	$((0.8 - 0.9)^2 + (0.8 - 0.7)^2) \div 2$	0.01
MSE de la imagen media	$((0.5 - 0.9)^2 + (0.5 - 0.7)^2) \div 2$	0.10

Lectura: con un solo número (z) PCA reduce el error de 0.10 a 0.01 porque aprovecha que los dos píxeles tienden a variar juntos. **Error común**: creer que PCA mira etiquetas; solo usa las imágenes.

En la clase (validación, 360 imágenes): imagen media 0.0729; PCA con 8 componentes 0.0235; autoencoder 64–8–64 0.0240 (nuestra reproducción) y autoencoder profundo 0.0193. PCA 8 reduce el error de la media en 67.8 % y el profundo, otro 17.7 % respecto de PCA 8. Pero una corrida, una semilla y una prueba ya conocida no establecen superioridad general.

Primer caso de cada dígito 0 a 5 en prueba (diapositiva 28) · se muestra recortado a [0, 1]; el MSE usa las salidas completas



Lo que debes recordar

- Imagen media: ignora la entrada; PCA: reconstrucción lineal con k números; autoencoder: no lineal
- Todas las referencias se ajustan solo con entrenamiento
- Menor MSE es mejor, pero una corrida no prueba superioridad

5 Arquitectura y conteo de parámetros: el latente no es el tamaño del modelo

Una arquitectura 64–8–64 significa 64 entradas, una capa latente de 8 unidades y 64 salidas. Cada capa tiene (entradas + 1) parámetros por unidad (el +1 es el sesgo). En 64–32–8–32–64 hay tres capas ocultas (32, 8 y 32). Las dos redes tienen un vector latente de 8 valores, pero la segunda aprende más pesos y sesgos porque tiene capas adicionales. Más parámetros dan más flexibilidad, más tiempo de cálculo y más posibilidad de memorizar.

Ejemplo resuelto · una red 64–16–64 (ejemplo propio) y la práctica de la diapositiva 53

Capa	Cálculo	Parámetros
64 → 16	16 · (64 + 1) = 16 · 65	1040
16 → 64	64 · (16 + 1) = 64 · 17	1088
Total 64–16–64	1040 + 1088	2 128
Diapositiva 53: 64–4–64	4 · 65 + 64 · 5 = 260 + 320	580
Diapositiva 53: 64–32–4–32–64	2 080 + (32·4 + 4) + (4·32 + 32) + 2 112 = 2 080 + 132 + 160 + 2 112	4 484

Lectura: bajar el latente de 8 a 4 reduce los parámetros de 1 096 a 580, pero reduce también lo que el decoder puede reconstruir. La práctica de la diapositiva 53 pide repetir con 4 y comparar el error de validación y los detalles de la imagen. **Error común**: creer que cuantos más parámetros, mejor reconstrucción en datos nuevos.



Sobre cada flecha: pesos + sesgos de esa capa (unidades × (entradas + 1)). Ambas redes resumen cada imagen en 8 valores; la profunda tiene más parámetros.

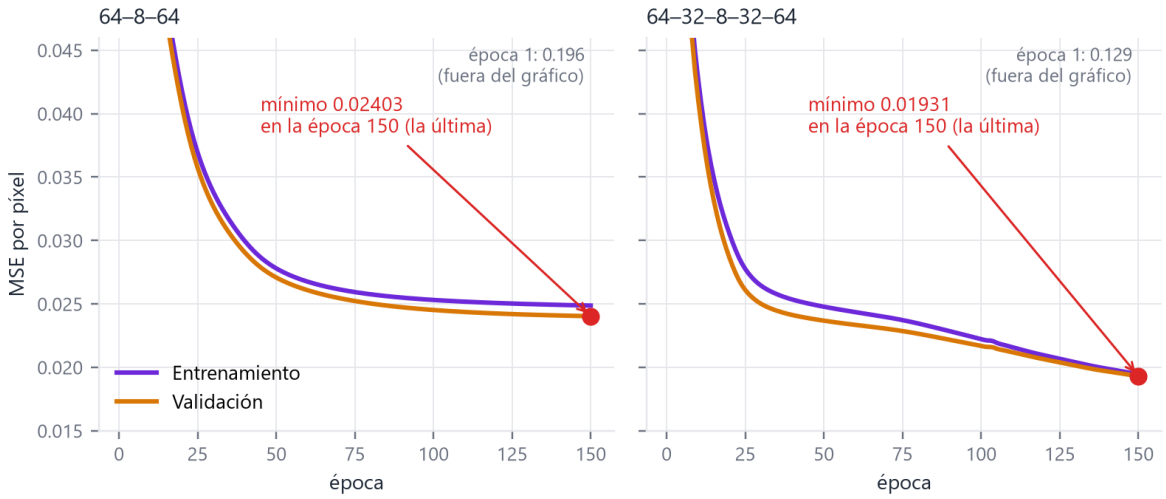
Lo que debes recordar

- Parámetros por capa = unidades × (entradas + 1)
- 64–8–64: 1 096; 64–32–8–32–64: 4 744
- Latente (8) y parámetros son cosas distintas

6 Cómo se entrena y cómo se elige: la validación y el límite de épocas

El entrenamiento repite cuatro pasos con cada lote de 64 imágenes: reconstruir ($x \to h \to \hat{x}$), medir $\ell(x, \hat{x})$, **retropropagar** los gradientes hacia el decoder y el encoder, y actualizar pesos y sesgos con Adam (tasa 0.002). Al terminar cada **época** (una pasada por las 1 077 imágenes de entrenamiento) se mide el MSE de validación *sin* actualizar parámetros, y se guarda una copia de la mejor época. La validación elige entre arquitecturas y épocas; la prueba se mira al final. El error que se grafica es el MSE por píxel sin la penalización alpha = 0.0001.

En la clase los dos mínimos de validación están en la época 150, la última: 64–8–64 llega a 0.02403 y 64–32–8–32–64 a 0.01931. Si el mínimo coincide con el límite de épocas, el presupuesto limita la conclusión: no sabemos si con más épocas seguiría mejorando. Y el programa recuerda que ese mínimo «no demuestra convergencia».



Lo que debes recordar

- Reconstruir → medir → retropropagar → actualizar → repetir; la validación solo compara
- Se elige arquitectura y época con validación; la prueba al final
- Un mínimo en la última época no demuestra convergencia

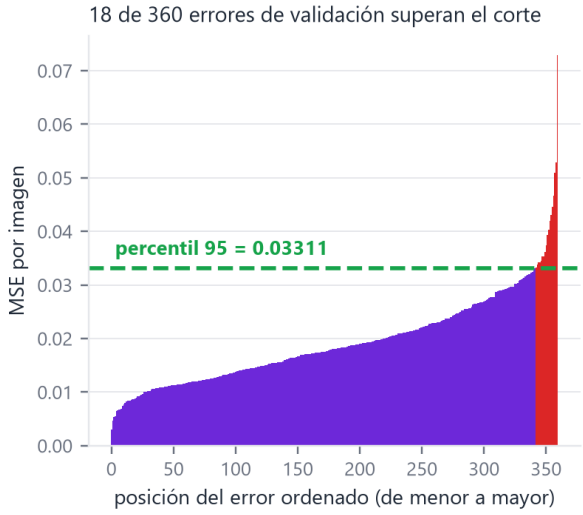
7 El error como alerta: puntaje, umbral y falsas alarmas

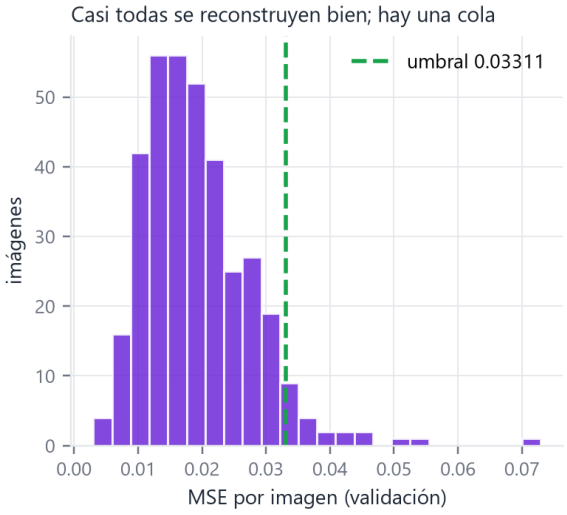
Si se entrena con lo habitual, un error de reconstrucción alto indica que una imagen no se parece a lo que el modelo aprendió: e(x) = MSE de x. Se declara una **alerta** si e(x) > c. El umbral c debe fijarse antes de mirar la prueba y con datos pertinentes; aquí se usa el **percentil 95** del error de validación. Esto hace que, por construcción, el 5 % de la validación quede sobre el corte. **No es un 95 % de confianza sobre defectos**; y e(x) no es una probabilidad de anomalía.

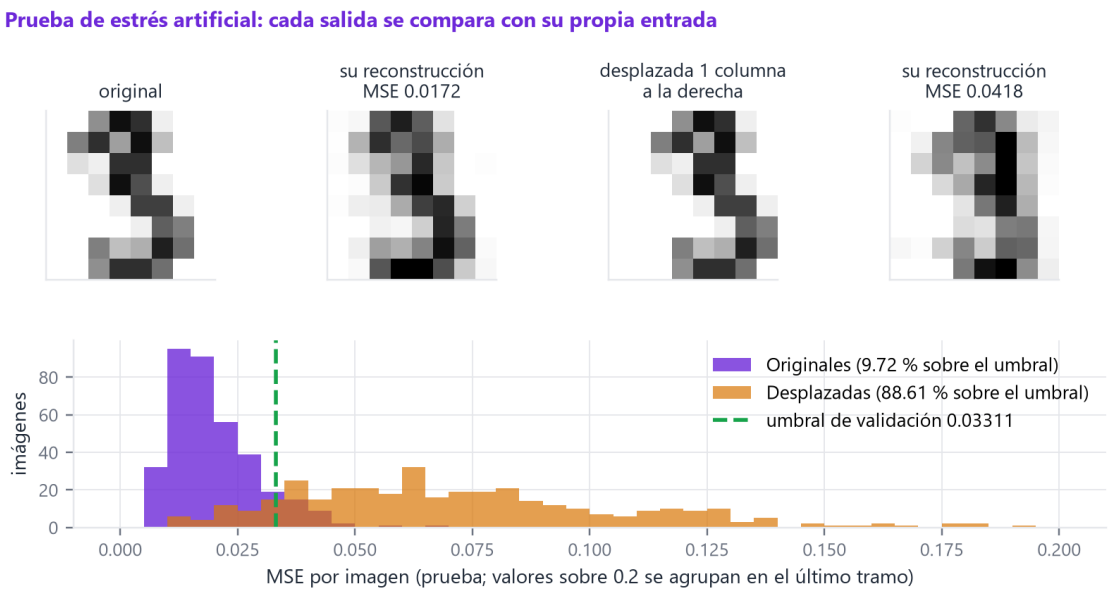
Ejemplo resuelto · percentil 80 de 10 puntajes (ejemplo propio)

Paso	Cálculo	Resultado
Puntajes ordenados	10 imágenes de validación	0.011, 0.014, 0.016, 0.018, 0.020, 0.022, 0.024, 0.027, 0.031, 0.058
Umbral c	percentil 80 (interpolación lineal): posición 0.8 · 9 = 7.2 entre 0.027 y 0.031	c = 0.027 + 0.2 · (0.031 − 0.027) = 0.0278
Alertas	puntajes mayores que c	2 de 10: 0.031, 0.058

Lectura: el umbral deja el 20 % superior por construcción, sin saber nada sobre defectos reales. Si una de esas alertas es una imagen perfectamente normal, es una **falsa alarma**; si un defecto real queda bajo el corte, es un **defecto no detectado**. Con el percentil 95 de la clase: c = 0.033108, 18 de 360 validaciones lo superan; en prueba, 35 de 360 originales (9.72 %) y 319 de 360 desplazadas (88.61 %). **Error común**: leer 0.058 como «5.8 % de probabilidad de defecto».







Cuándo puede fallar una alerta (diapositiva 42): si hay variación habitual que el modelo no representó, avisará sobre casos aceptables; si hay anomalías en el entrenamiento y el decoder es muy flexible, puede aprender a reconstruirlas; si cambia la forma de adquirir las imágenes, el puntaje refleja el cambio técnico. Por eso se necesita una validación específica: definir los defectos relevantes, separar por lote o fecha, calibrar el umbral con datos pertinentes, evaluar en una reserva, examinar errores y prever revisión humana antes de automatizar (diapositiva 46).

Lo que debes recordar

- Alerta: $e(x) > c$; c se fija con datos pertinentes antes de mirar la prueba
- El percentil de un puntaje no es un nivel de confianza sobre defectos
- Una alerta no confirma un defecto; un puntaje no es una probabilidad

8 Glosario rápido

Término	Qué es
Representación	Descripción numérica de un objeto (64 intensidades, o 8 coordenadas latentes).
Autoencoder	Red entrenada para reproducir su entrada pasando por un resumen pequeño.
Encoder f, decoder g	$h = f(x)$ resume; $\hat{x} = g(h)$ reconstruye; con parámetros θ y ψ.
Vector latente h	Salida de la capa central; aquí 8 coordenadas entre −1 y 1 (tanh).
Reconstrucción $\hat{x}$	Estimación de la entrada; mismo tamaño y significado que x.
MSE de reconstrucción	Promedio de $(\hat{x}_j - x_j)^2$; ℓ por imagen, L por conjunto.
Cuello de botella	Restricción que obliga a resumir (aquí, 8 coordenadas); sin ella, la identidad copia todo.
PCA con k componentes	Reconstrucción lineal con k números: $\hat{x} = \bar{x} + (x - \bar{x})P_k P_k^T$.
Capa latente y parámetros	Latente = cuántos números resumen la imagen; parámetros = pesos y sesgos de toda la red.
Época, lote, Adam	Pasada completa, grupo de 64 imágenes, optimizador (tasa 0.002).
Puntaje de anomalía	$e(x)$ = MSE de la imagen; no es una probabilidad.
Umbral c	Corte a partir del cual se alerta; aquí el percentil 95 de validación.
Falsa alarma	Alerta sobre una imagen aceptable; el reverso es un defecto no detectado.